Cirrhosis: Diagnosing Disease in Liver Patients through ML

Angela Meyer

ITSY 1342: Information Technology Security

Dr. Phillip Davis

May 4, 2026

Abstract

This project explores whether a basic neural network can predict patient outcomes for cirrhosis using standard clinical measurements. I used the cirrhosis.csv dataset from OpenStax, which contains 418 patient records with demographic, biochemical, and disease-stage features. After dropping rows with missing values, I was left with 276 complete cases. I removed the ID, N_Days, Status, and Drug columns, one-hot encoded the Edema variable, and converted the remaining categorical fields (Sex, Ascites, Hepatomegaly, Spiders) to binary values. The Status column was reduced to a binary target, with 0 representing patient death and 1 representing survival during the trial. I scaled the features with StandardScaler and split the data 80/20 for training and testing. The model was built in TensorFlow as a Sequential network with two hidden layers of 16 neurons using ReLU activation, a sigmoid output layer, the Adam optimizer, and binary cross-entropy loss, trained over 10 epochs. The final training loss was 0.54 with a training accuracy of 77.7 percent, and the model reached 80.4 percent accuracy on the test set. The results suggest the model captures meaningful patterns but is not yet reliable enough for real clinical diagnosis.

Keywords: cirrhosis, neural networks, TensorFlow, binary classification, clinical prediction, machine learning

---

Introduction

This research applies introductory deep learning techniques to a well-known clinical dataset originally collected during the Mayo Clinic primary biliary cirrhosis trial (Dickson et al., 1989). The primary objectives are to (1) clean and prepare the clinical dataset for modeling, (2) encode mixed categorical and numerical features into a format suitable for a neural network, and (3) build a binary classification model in TensorFlow that predicts whether a patient survived during the trial period.

Data Source and Description

Dataset Selection

Data were obtained from the OpenStax *Principles of Data Science* companion materials (OpenStax, 2024), which distribute a cleaned version of the Mayo Clinic primary biliary cirrhosis (PBC) dataset. The original study followed 418 patients diagnosed with PBC between 1974 and 1984 and recorded demographic information, biochemical lab values, and clinical signs of disease progression (Dickson et al., 1989).

Data Acquisition

The dataset was downloaded as a CSV file from the OpenStax companion link (https://openstax.org/r/datach7) and loaded directly into a pandas DataFrame for analysis. The raw file contained 418 patient records across 20 columns, including identifiers, trial duration, drug assignment, demographic details, lab values, and clinical observations.

Sample Data Structure

ID	Status	Age	Sex	Bilirubin	Albumin	Platelets	Prothrombin	Stage
1	D	21464	F	14.5	2.60	190.0	12.2	4.0
2	C	20617	F	1.1	4.14	221.0	10.6	3.0
3	D	25594	M	1.4	3.48	151.0	12.0	4.0

Methodology

Data Cleansing

The raw dataset required several preprocessing steps before it could be used to train a neural network:

1. **Missing Value Treatment:** A total of 142 rows contained missing values across columns such as Cholesterol, Copper, Triglycerides, and the binary clinical signs. These rows were removed using dropna(), leaving a working dataset of 276 complete records.
2. **Column Removal:** The ID, N_Days, Status, and Drug columns were dropped from the feature set. ID carries no predictive value, N_Days leaks survival information, Status is the target variable, and Drug was excluded per the project specification.
3. **Categorical Encoding:** The Edema variable, which had three possible values (N, Y, S), was converted using one-hot encoding through pd.get_dummies(). Sex was mapped from

F and M to 1 and 0, and the Ascites, Hepatomegaly, and Spiders columns were mapped from Y and N to 1 and 0.

4. **Target Variable:** Status was converted to a binary outcome where 0 represents patient death during the trial and 1 represents survival or censored status.

[2] 0s

```python
import pandas as pd

#Load the dataset
df = pd.read_csv('cirrhosis.csv')
```

[3] 5s

```python
import numpy as np
import tensorflow as tf

np.random.seed(42)
tf.random.set_seed(42)
```

[4] 0s

```python
#preview before cleaning data
print("Data before cleaning:"), df.shape
print(df.isnull().sum())
```

```
Data before cleaning:
ID                 0
N_Days             0
Status             0
Drug             106
Age                0
Sex                0
Ascites          106
Hepatomegaly     106
Spiders          106
Edema              0
Bilirubin          0
Cholesterol      134
Albumin            0
Copper           108
Alk_Phos         106
SGOT             106
Tryglicerides    136
Platelets         11
Prothrombin        2
Stage              6
dtype: int64
```

[5] 0s

```python
#Remove all rows with missing data
df = df.dropna()
```

[6] 0s

```python
#confirm result
print("\nShape after removing missing data:", df.shape)
print(df.head())
```

```
Shape after removing missing data: (276, 20)
   ID  N_Days Status             Drug    Age Sex Ascites Hepatomegaly Spiders  \
0   1     400      D  D-penicillamine  21464   F       Y            Y       Y
1   2    4500      C  D-penicillamine  20617   F       N            Y       Y
2   3    1012      D  D-penicillamine  25594   M       N            N       N
3   4    1925      D  D-penicillamine  19994   F       N            Y       Y
4   5    1504     CL          Placebo  13918   F       N            Y       Y

  Edema  Bilirubin  Cholesterol  Albumin  Copper  Alk_Phos    SGOT  \
0     Y       14.5        261.0     2.60   156.0    1718.0  137.95
1     N        1.1        302.0     4.14    54.0    7394.8  113.52
2     S        1.4        176.0     3.48   210.0     516.0   96.10
3     S        1.8        244.0     2.54    64.0    6121.8   60.63
4     N        3.4        279.0     3.53   143.0     671.0  113.15

   Tryglicerides  Platelets  Prothrombin  Stage
0          172.0      190.0         12.2    4.0
1           88.0      221.0         10.6    3.0
2           55.0      151.0         12.0    4.0
3           92.0      183.0         10.3    4.0
4           72.0      136.0         10.9    3.0
```

```
[7]
#drop columns, ID, N_Days, Status, Drug
X = df.drop(columns=['ID', 'N_Days', 'Status', 'Drug'])
display(X.head())
```

	Age	Sex	Ascites	Hepatomegaly	Spiders	Edema	Bilirubin	Cholesterol	Albumin	Copper	Alk_Phos	SGOT	Tryglicerides	Platelets	Prothrombin	Stage
0	21464	F	Y	Y	Y	Y	14.5	261.0	2.60	156.0	1718.0	137.95	172.0	190.0	12.2	4.0
1	20617	F	N	Y	Y	N	1.1	302.0	4.14	54.0	7394.8	113.52	88.0	221.0	10.6	3.0
2	25594	M	N	N	N	S	1.4	176.0	3.48	210.0	516.0	96.10	55.0	151.0	12.0	4.0
3	19994	F	N	Y	Y	S	1.8	244.0	2.54	64.0	6121.8	60.63	92.0	183.0	10.3	4.0
4	13918	F	N	Y	Y	N	3.4	279.0	3.53	143.0	671.0	113.15	72.0	136.0	10.9	3.0

```
[8]
#Convert text to numerical data - One-hot encoding

X = pd.get_dummies(X, columns=['Edema'])
```

```
[9]
#use lambda function to apply a simple rule to each value
y = df['Status'].apply(lambda x: 0 if x == 'D' else 1)
display(y.head())
```

	Status
0	0
1	1
2	0
3	0
4	1

dtype: int64

Feature Scaling and Train/Test Split

The cleaned dataset was split into 80% training and 20% testing using train_test_split with a fixed random state of 42 so the split is the same every run. The features in this dataset cover very different scales (Age is recorded in days, Bilirubin in mg/dL, and Alkaline Phosphatase in much larger units), so I applied StandardScaler to put them all on the same footing. Without scaling, the larger-valued features would carry more weight in training and slow the model down.

```
[11]
#split 276 rows into 80% training and 20% testing. random_state=42 locks the shuffle so it is the same split each time you run it
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
[12]
from sklearn.preprocessing import StandardScaler

#scale the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Machine Learning Model

A feedforward neural network was built using the TensorFlow Keras Sequential API. The model has an input layer that matches the number of features, two hidden layers of 16 neurons each

using ReLU activation, and a single output neuron with a sigmoid activation that returns a probability between 0 and 1 (Goodfellow et al., 2016). Any patient with an output above 0.5 is classified as a survivor, and anyone at or below 0.5 is classified as deceased. The model was compiled with the Adam optimizer and binary cross-entropy loss, then trained for 10 epochs.

```
[13]
✓ 1s
import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Input(shape=(X_train.shape[1],)), #tells model how many input featues to expect
    keras.layers.Dense(16, activation='relu'), #2 hidden layers with 16 neurons each
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid') #squishes output to value of 0 and 1
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

#epoch: how many times the model passes through the traning set
history = model.fit(X_train, y_train, epochs=10, verbose=1)
```

```
Epoch 1/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.3545 - loss: 0.8280
Epoch 2/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.3909 - loss: 0.7788
Epoch 3/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.4500 - loss: 0.7382
Epoch 4/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.4727 - loss: 0.7031
Epoch 5/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5773 - loss: 0.6733
Epoch 6/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.6636 - loss: 0.6474
Epoch 7/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7227 - loss: 0.6244
Epoch 8/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7318 - loss: 0.6038
Epoch 9/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7455 - loss: 0.5854
Epoch 10/10
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7682 - loss: 0.5690
```

Results

Descriptive Statistics

After cleaning, the working dataset contained 276 patients with 19 input features after one-hot encoding. The Status target was reasonably balanced between survivors (Status = 1) and

deceased patients (Status = 0), which means accuracy is a meaningful metric without needing to oversample or weight the classes.

Training Performance

The model improved steadily across all 10 epochs. Training accuracy started near 60% and finished at 77.7%, while loss dropped from 0.67 down to 0.54. The trend suggests the model was still learning when training stopped, but most of the gains had already been captured.

[14]
✓ 0s

```
from sklearn.metrics import accuracy_score

final_loss = history.history['loss'][-1] #grabs the value from the last epoch
final_acc = history.history['accuracy'][-1]

y_pred = (model.predict(X_test) > 0.5).astype(int).flatten()
test_acc = accuracy_score(y_test, y_pred)

print(f"Final Training Loss: {final_loss}")
print(f"Final Training Accuracy: {final_acc}")
print(f"Final Testing Accuracy: {test_acc}")
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
Final Training Loss: 0.5689813494682312
Final Training Accuracy: 0.7681818008422852
Final Testing Accuracy: 0.7857142857142857
```

Model Performance on Test Set

The trained model was evaluated on the held-out test set of 56 patients. Final results were:

- **Final Training Loss:** 0.538
- **Final Training Accuracy:** 77.7%
- **Test Accuracy:** 80.4%

Test accuracy slightly exceeded training accuracy, which suggests the model is not overfitting on this run and that the 20% test split happened to contain patients whose outcomes were a bit easier to predict.

Discussion

Scientific Interpretation

A neural network learns by adjusting the weights connecting its layers so that its predictions get closer to the true labels over time. For binary classification, the model uses a loss function called binary cross-entropy, which essentially measures how confident and how correct each prediction is. The model is rewarded when it assigns high probability to the right class and penalized when it assigns high probability to the wrong one. Each training pass updates the weights to reduce that overall penalty.

The Adam optimizer handles those weight updates using gradient descent with adaptive learning rates, and the ReLU activation in the hidden layers lets the network model nonlinear relationships between clinical variables and survival outcome (Goodfellow et al., 2016). A test accuracy of 80.4% means roughly four out of every five patients in the test set were correctly classified, which is a reasonable result for a small, mixed-feature clinical dataset.

Limitations

There are a few limitations worth pointing out. Dropping every row with a missing value cut the dataset from 418 patients down to 276, which threw away a lot of usable information and may have skewed the sample toward patients with more complete records. The model also runs for only 10 epochs with no validation set, so I cannot tell exactly when overfitting would start to happen. The network itself is very simple, with only two hidden layers and no dropout, regularization, or hyperparameter tuning. Finally, the feature set leaves out the assigned drug (D-penicillamine versus placebo), which removes a clinical variable that could matter for survival.

Clinical Implications

While 80% accuracy is encouraging for a classroom project, it is far below the threshold needed for actual clinical deployment. A model used to support cirrhosis diagnosis or prognosis would need higher sensitivity to avoid missing patients at risk of death, and it would also need to report calibrated probabilities along with confidence intervals, not just a binary prediction. The current model is best understood as a proof of concept showing that even a small neural network can learn meaningful patterns from routine lab values, not as a tool ready for use in patient care.

Conclusion

This project demonstrated a complete supervised learning workflow applied to clinical data, from cleaning and encoding the cirrhosis dataset through training and evaluating a TensorFlow neural network. The final model reached 77.7% training accuracy and 80.4% test accuracy after 10 epochs, indicating that even a shallow network can extract useful patterns from standard clinical measurements. Future work should incorporate missing value imputation rather than row removal, add a validation split with early stopping, explore deeper architectures or alternative algorithms such as random forests, and include additional clinical features such as drug assignment. The exercise reinforced the importance of careful preprocessing and the realistic limits of small datasets in healthcare machine learning.

References

Anthropic. (2026). *Claude* (claude-sonnet-4-6) [Large language model]. https://www.anthropic.com

Dickson, E. R., Grambsch, P. M., Fleming, T. R., Fisher, L. D., & Langworthy, A. (1989). Prognosis in primary biliary cirrhosis: Model for decision making. *Hepatology*, *10*(1), 1–7. https://doi.org/10.1002/hep.1840100102

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

OpenStax. (2024). *Principles of data science*. Rice University. https://openstax.org/details/books/principles-data-science\

World Health Organization. (2023). *Hepatitis and liver disease*. https://www.who.int/health-topics/hepatitis